

CHAPTER 2

Problem Solving

By: Ashok Basnet

Contents

2

- Steps in Problem Solving
- Defining Problems as a State Space Search
- Problem Formulation
- Problem Characteristics
- Problem Types
 - ▣ Well Defined Problems
 - ▣ Constraint Satisfaction Problems
- Production Systems
- Game Playing

Steps in Problem Solving

3

- ❑ **Goal Formulation:** Based on current situation and the agent's performance measure.
- ❑ **Problem Formulation:** The process of deciding what actions and states to consider given a goal.
- ❑ **Searching:** Looking for a sequence of actions that reaches the goal
- ❑ **Execution:** Recommended action after solution is found.

Components to formulate problem

4

- **Initial State:** This state requires an initial state for the problem which starts the AI agent towards a specified goal. In this state new methods also initialize problem domain solving by a specific class.
- **Action:** This stage of problem formulation works with function with a specific class taken from the initial state and all possible actions done in this stage.
- **Transition:** This stage of problem formulation integrates the actual action done by the previous action stage and collects the final stage to forward it to their next stage.

Components to formulate problem

5

- **Goal test:** This stage determines that the specified goal achieved by the integrated transition model or not, whenever the goal achieves stop the action and forward into the next stage to determines the cost to achieve the goal.
- **Path costing:** This component of problem-solving numerical assigned what will be the cost to achieve the goal. It requires all hardware software and human working cost.

State Space Search

6

- A **state space** is the set of all possible configurations of a system.
- It is a useful abstraction for reasoning about the behaviour of a given system and is widely used in the fields of artificial intelligence and game theory.
- **State space search** is a process used in the field of computer science, including artificial intelligence (AI), in which successive configurations or states of an instance are considered, with the intention of finding a goal state with the desired property.
- Problems are often modelled as a state space, a set of states that a problem can be in. The set of states forms a graph where two states are connected if there is an operation that can be performed to transform the first state into the second.

Defining Problems as a State Space Search

7

- Things that must be done to solve a problem are:
 - ▣ **Define the problem**: It includes precise specifications of what the initial situation(s) will be as well as what final situations constitute acceptable solutions to the problem
 - ▣ **Analyse the problem**: Few very important aspects those having immense impact on the appropriateness of various possible techniques for problem solving is to be critically examined
 - ▣ **Isolate and Represent the task knowledge**: Knowledge necessary to solve the problem must be identified, isolated and represented
 - ▣ **Choose and apply the best technique**: Among the alternatives identify the best technique to solve the problem and apply it

Defining Problems as a State Space Search

8

- ❑ **Problem Statement:** Play Chess, Solve Tower of Hanoi
 - ▣ To develop the solution to the problem, problem must be well identified
 - ▣ Starting with specification of states, rules and outcomes
 - ▣ Focusing on Solving the Problem i.e. identifying the GOAL
- ❑ Incomplete and Complete Problem Statement
- ❑ Complete Set of Rules is a must
 - ▣ Example: If White pawn at sqr (row e, column 2) is true and White pawn at sqr (row e, column 3) is empty then Move pawn from sqr (row e, column 2) to sqr(row e, column 3) → its specific way of writing rule
 - ▣ Developing such rules is very tedious as chess game alone will have 10^{120} rules
 - ▣ So, generalization is to be done

Defining Problems as a State Space Search

9

- Problem
- State
- **State Space:** where each state corresponds to a stable situation
 - ▣ Initial State
 - ▣ Rules for transition from one state to another
 - ▣ Final State → Ultimate Goal
- State Space Representation → forms the basis for AI methods
- State Space Structure corresponds to the structure of problem solving in two ways
 - ▣ It allows for a formal definition of the problem
 - ▣ It permits to define the process of solving the particular problem as a combination of known techniques and searching mechanism

Defining Problems as a State Space Search: An Example – A Water Jug Problem

10

Problem

You are given two jugs, a 4 litre one and a 3 litre one. None of them have any measuring markers on it. There is a pump that can be used to fill the jugs with water. How can you get exactly a 2 litres of water in to the 4 litre jug?

Defining Problems as a State Space Search:

An Example – A Water Jug Problem

11

State Space

- Set of ordered pair of integers (x, y) , such that $x = 0, 1, 2, 3, \text{ or } 4$ and $y = 0, 1, 2, \text{ or } 3$, where x represents the quantity of water in 4 litre jug and y represents the quantity of water in 3 litre jug
- The start state is $(0, 0)$
- The final state is $(2, n)$

Defining Problems as a State Space Search:

An Example – A Water Jug Problem → Rules

12

1. (x, y) if $x < 4 \rightarrow (4, y)$
Fill the 4 litre jug
2. (x, y) if $y < 3 \rightarrow (x, 3)$
Fill the 3 litre jug
3. (x, y) if $y > 0 \rightarrow (x, 0)$
Empty the 3 litre jug
4. (x, y) if $x > 0 \rightarrow (0, y)$
Empty the 4 litre jug
5. (x, y) if $x > 0 \rightarrow (x-d, y)$
Put some water out of 4 l jug
6. (x, y) if $y > 0 \rightarrow (x, y-d)$
Put some water out of 3 l jug
7. (x, y) if $x+y \geq 4$ & $y > 0 \rightarrow (4, y-(4-x))$
Put water from 3l jug into 4l jug to fill it
8. (x, y) if $x+y \geq 3$ & $x > 0 \rightarrow (x-(3-y), 3)$
Put water from 4l jug into 3l jug to fill it
9. (x, y) if $x+y < 3$ & $x > 0 \rightarrow (0, x+y)$
Put all the water from 4l jug in to 3l jug
10. (x, y) if $x+y < 4$ & $y > 0 \rightarrow (x+y, 0)$
Put all the water from 3l jug in to 4l jug

Defining Problems as a State Space Search:

An Example – A Water Jug Problem → Solution

13

Water Quantity in 4 l Jug	Water Quantity in 3 l Jug	Rule Applied
0	0	Initial Phase
0	3	2
3	0	10
3	3	2
4	2	7
0	2	4
2	0	10

Defining Problems as a State Space Search

14

- Solve (Goose, Grain, Fox, Man) problem.
- Solve (Missionary, cannibal) Problem

Defining Problems as a State Space Search

15

□ Operationalization

- ▣ It's the process of creation of a formal and manipulability description of the problem
- ▣ It is the first step toward the design of a program to solve a problem
- ▣ We expect to create such programs which can themselves produce formal descriptions of the problems from the informal ones

Problem Formulation

16

→ Process of deciding what **action and states** to consider, given a goal.

For providing a formal description of a problem, the things to be done are:

- ❑ Define a state space that contains all the possible configuration of the relevant objects.
- ❑ Specify one or more states within the space that describe possible situations from which the problem-solving process may start, known as the initial states.
- ❑ Specify one or more states that would be acceptable as solutions to the problem, known as the goal states or final states.
- ❑ Specify a set of rules that describe the actions (operators) available. While doing these, the following issues are to be highlighted:
 - ▣ What unstated assumptions are present in the informal problem description?
 - ▣ How much the rules should be generalized?
 - ▣ How much of the work required solving the problem should be pre- computed and represented in the rules.

Problem Characteristics

17

- What is the possibility in terms of **decomposability**?
- Can **solution be ignored or undone** or is solution **Irrecoverable**?
- Is the universe of the problem is **predictable**?
- Is a good solution to the problem is **absolute or relative**?
- Is the **solution a path or a state**?
- What is the **role of knowledge in solving a problem using artificial intelligence**?
- Does the task of solving a problem **require human interaction**?

Problem Types

18

- Well Defined Problems
- Constraint Satisfaction Problems

Problem Types: Well Defined Problem

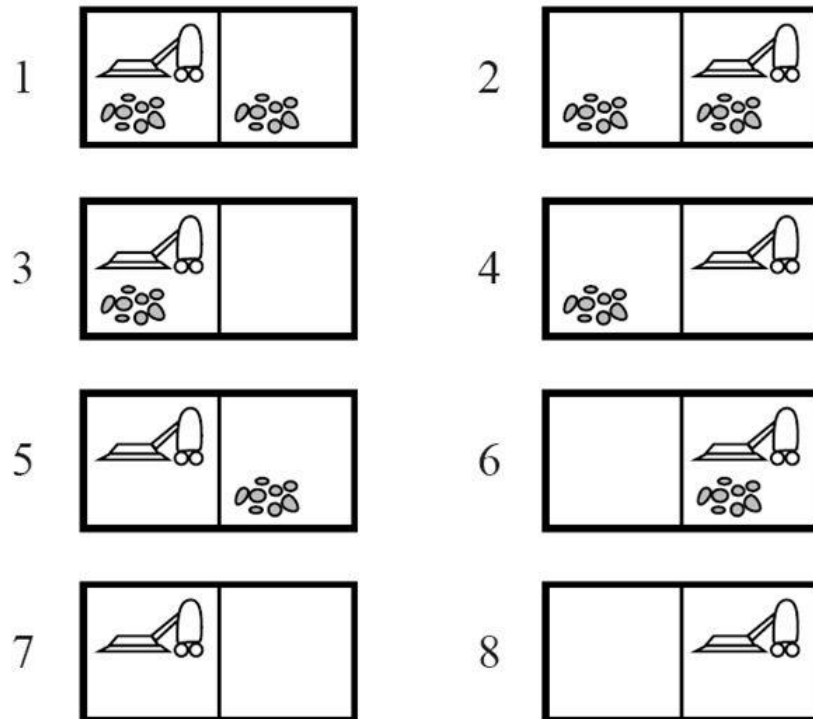
19

- Major Components of a Problem
 - ▣ **Initial State**
 - ▣ **Actions** available to the agent in the **state space**
 - ▣ **Goal Test** determining the given state as a goal state
 - ▣ **Path Cost** function that assigns numeric value to each path considering the **step cost** as well
- If all these components are completely defined along with the optimal solution for the problem, then such problem is a well defined problem

Problem Types: Well Defined Problem

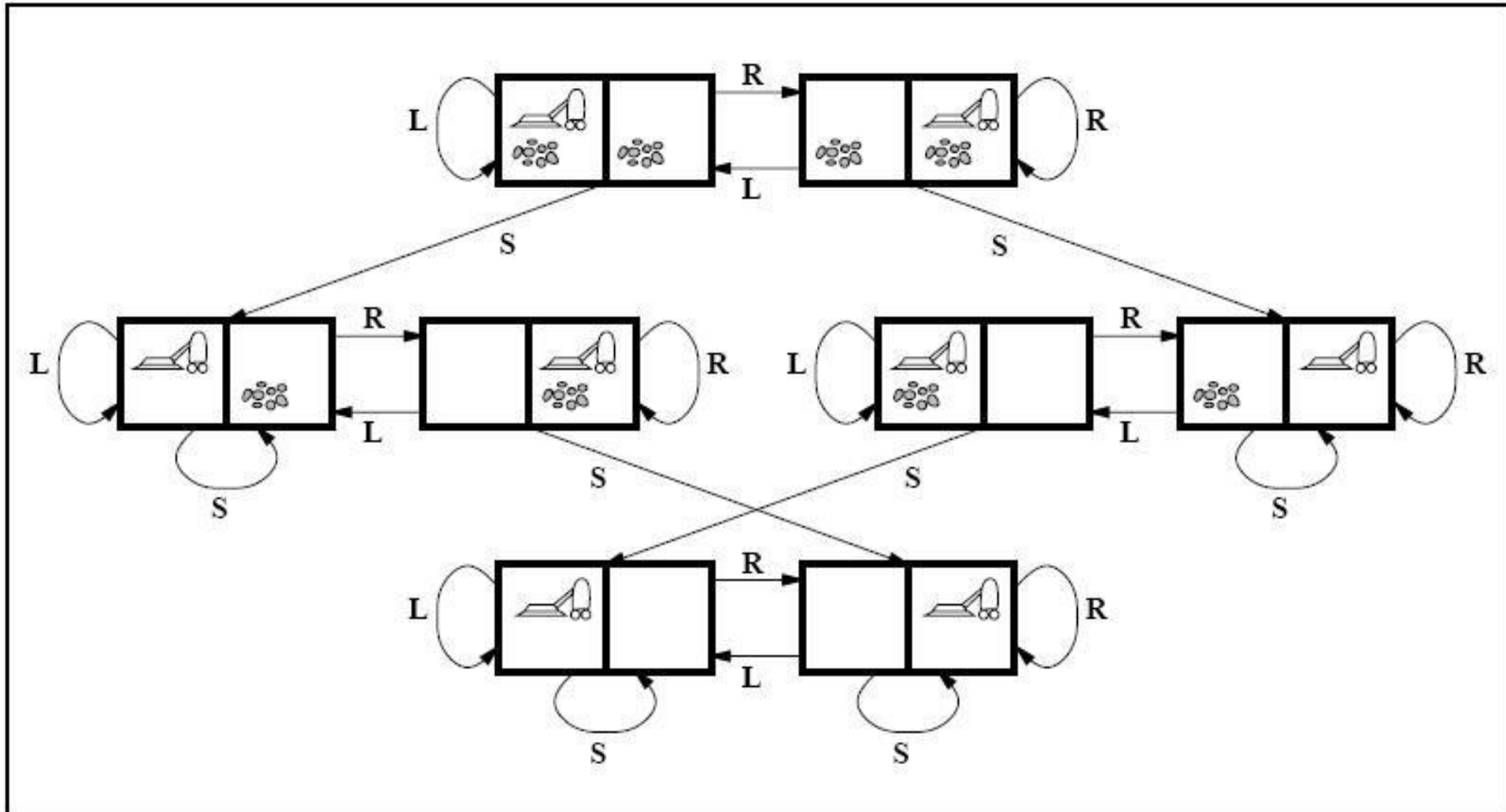
20

The eight possible states in the vacuum world



Problem Types: Well Defined Problem

21



Problem Types: Well Defined Problem

22

- Generalization of actions and rules
- Specialization of actions and rules

Generalization of Actions and Rules:

23

- **Actions:** Generalization of actions refers to the ability to apply a certain action or set of actions across a broad range of situations or contexts. For example, if you have a general action like "problem-solving," it can be applied to various problems in different domains.
- **Rules:** Generalization of rules involves creating principles or guidelines that are applicable in diverse scenarios. These rules are formulated to be broad enough to cover a range of situations. In artificial intelligence or machine learning, generalization is a key concept where models are trained to perform well on unseen data.

Generalization

24

- Machine Learning Classification Model:
 - **Action:** General action is "classification" (assigning labels to data points).
 - **General Rule:** The model is trained to recognize patterns and generalize its learning to correctly classify new, unseen data.
- Educational Strategies:
 - **Action:** General action is "teaching problem-solving skills."
 - **General Rule:** Students are taught general problem-solving strategies that can be applied across various subjects and real-life situations.

Specialization of Actions and Rules

25

- **Actions:** Specialization of actions involves tailoring specific actions to particular situations or contexts. Instead of having a general problem-solving action, you might have specialized problem-solving strategies for different types of problems.
- **Rules:** Specialization of rules means creating specific guidelines or principles for particular scenarios. These rules are designed to address the nuances of a specific situation rather than being broadly applicable. In some cases, specialization can lead to more efficient and effective solutions.

Specialization

26

- Medical Diagnosis System:
 - **Action:** Specialized action is "diagnosing a specific medical condition."
 - **Specialized Rule:** Different rules and algorithms are designed for the diagnosis of specific diseases based on unique symptoms and test results.
- Legal System:
 - **Action:** Specialized action is "resolving legal disputes."
 - **Specialized Rule:** Different rules and legal precedents apply to different types of cases, and the legal system often relies on specialization for handling specific areas of law.

Balance

27

- Self-Driving Cars:
 - **Generalization:** The car needs to generalize actions like "obstacle avoidance" across different environments.
 - **Specialization:** In certain situations, specialized actions like "pedestrian crosswalk recognition" or "traffic signal interpretation" are needed.
- Personalized Marketing:
 - **Generalization:** Marketing strategies may involve general actions like "targeting a specific demographic."
 - **Specialization:** Personalized recommendations are specialized actions that require specific rules based on individual preferences and behavior.

Constraint Satisfaction Problem

28

- **Variables:** Represent elements of the problem that need to be assigned values.
- **Domains:** Each variable has a domain, which is the set of possible values it can take.
- **Constraints:** Define relationships or restrictions on the variables. Constraints specify which combinations of values for subsets of variables are allowed
- In a scheduling CSP, a constraint could be that two tasks cannot be scheduled for the same time.

Problem Types: Constraint Satisfaction Problem

29

- **Constraint satisfaction problems (CSPs)** are mathematical questions defined as the set of objects whose state must satisfy a number of constraints or limitations
- A search procedure that operates in a space of constraint sets
- **Constraints** are discovered and propagated as far as possible throughout the system
- It (Search) continues until the solution is achieved
- An assumption is made about something and added as a new constraint

Problem Types: Constraint Satisfaction Problem

30

- Constraint Satisfaction Problem consists of variables with constraints on them
- Many important real world problems can be described as CSPs
- The structure of a CSP can be represented by constraint graph
- CSP has states and goal test conformed to a standard, structured and very simple representation
- Simply, CSP is such that the goal is to discover some problem state that satisfies a given set of constraints

Problem Types: Constraint Satisfaction Problem

31

- A CSP is defined by a set of **variables**, $X_1, X_2, \dots, X_n$, and a set of constraints, $C_1, C_2, \dots, C_m$. Each variable X_i has a nonempty domain D_i of possible values. Each constraint C_i involves some subset of the variables and specifies the allowable combinations of values for that subset. A state of the problem is defined by an assignment of values to some or all of the variables, $\{X_i=V_i, X_j=V_j, \dots\}$.

Intelligent Backtracking

32

- $V = \{1, 2, 3, 4\}$
- $C = \{ 1 \neq 2, 1 \neq 3, 1 \neq 4, 2 \neq 4, 3 \neq 4 \}$
- $D = \{ \text{Red, Green, Blue} \}$
- Apply intelligent backtracking to solve the problem.

Intelligent Backtracking

33

- **Assignment:** { }
- **Select Variable:** 1
- **Order Domain Values:** {Red, Green, Blue}
- **Check Constraints for Red:**
 - ▣ Satisfies $1 \neq 2, 1 \neq 3, 1 \neq 4, 2 \neq 4, 3 \neq 4$
- **Recursive Call with** {1:Red}
- **Continue with the next unassigned variable, 2**
- **Order Domain Values for 2:** {Red, Green, Blue}
- **Check Constraints for Red:**
 - ▣ Violates $1 \neq 2$ (backtrack)
- **Check Constraints for Green:**
 - ▣ Satisfies $2 \neq 4, 3 \neq 4$
- **Recursive Call with** {1:Red, 2:Green}

Problem Types: Constraint Satisfaction Problem

34

- Constraint propagation terminates for one of the two reasons:
 - ▣ Contradiction detected: No solution consistent with known constraints
 - ▣ Propagation has run of stream and there are no further changes that can be made on the basis of current knowledge
- Variables: Discrete and Finite Domain
- Constraints: Linear and Non Linear

Problem Types: Constraint Satisfaction Problem

35

- Types of Constraints
 - ▣ Unary: Restricts the value of single variable
 - ▣ Binary: Two variables
 - ▣ Higher order constraints: More than two variables

Production System

36

□ Definition of Production:

- In the context of production systems, a "production" refers to a rule or a set of rules that define the behavior of the system.
- Each rule typically follows an IF-THEN structure, where certain conditions (IF) trigger specific actions or outcomes (THEN).
- The collective set of rules forms the knowledge base of the production system.

Production System

37

□ Enhancing Creation of AI-Based Programs:

- Production systems provide a structured and rule-based approach to programming in AI.
- They allow developers to define a set of rules and conditions that guide the behavior of an AI program.
- This rule-based paradigm is particularly useful for representing knowledge and encoding decision-making processes.

□ Automation of Machines:

- Production systems facilitate the automation of tasks and processes in machines and AI systems.
- By specifying rules and conditions, these systems can autonomously make decisions and take actions without direct human intervention.
- Automation improves efficiency, reduces manual effort, and enables machines to operate in various environments.

Production System

38

□ Resolution of Conflicts:

- IF-THEN conditions in production systems enable the resolution of conflicts in automated processes.
- Conflicts can arise when multiple rules are applicable, and the system needs a mechanism to decide which rule to prioritize.
- The use of rules and conditions allows for the systematic resolution of conflicts based on predefined criteria.

□ Action Selection and Automated Planning:

- Production systems enhance action selection by providing a mechanism for the system to choose appropriate actions based on the current state and input conditions.
- Automated planning involves the system autonomously generating sequences of actions to achieve a goal, and production systems play a crucial role in this process.

Global database

39

- ❑ This contains the architecture of the production system and acts as a central data structure.
- ❑ All the information and data used in the execution of tasks is stored in this database.
- ❑ The production rules of the system operate here.
- ❑ There are two types of global databases: temporary and permanent.
- ❑ The **temporary global database** consists of short-term actions that are based on circumstances.
- ❑ In a **permanent global database**, there are fixed actions that cannot be altered.

Production System Components

40

- A production system consists of three main components:
- **A global database**
- **A set of rules**
- **A control system.**

A global database

41

- The global database, often referred to as the **working memory or the data store**, is a repository of information that the production system utilizes during its operation.
- It holds the **current state of the system**, including facts, variables, and other relevant data.
- The content of the global database is constantly updated as the system processes rules and makes decisions.

Production rules

42

- The set of rules defines the knowledge and logic of the production system. Each rule typically follows an **IF-THEN structure**.
- **IF-Condition:** Describes a condition or a set of conditions that are evaluated against the contents of the global database.
- **THEN-Action:** Specifies the action or set of actions to be taken if the IF-Condition is satisfied.
- Rules collectively represent the knowledge and decision-making capabilities of the system. They guide the system's behavior based on the conditions present in the global database.

A control system

43

- ❑ The control system, also known as the inference engine, is responsible for managing the flow of the production system.
- ❑ It determines which rules to fire based on the current state of the global database.
- ❑ The control system processes the rules in a systematic manner, using strategies such as forward chaining (data-driven) or backward chaining (goal-driven) to achieve the system's objectives.
- ❑ The control system orchestrates the interaction between the global database and the set of rules, ensuring that the rules are applied appropriately to reach desired outcomes.

Examples: Production System

44

- In a washing machine, the condition-action rules enable the system to allow water and wash clothes based on the size and nature of clothes. If the size is large, the system will take the action of allowing large amounts of water. If the clothes are greasy, the machine will wash them for a long period.
- In autonomous cars, the production system uses the IF-THEN conditions to avoid collision with objects or other cars. The following shows examples of how these conditions are applied in autonomous cars.
 - ▣ IF there is an object or car in front, THEN apply brakes.
 - ▣ IF there is a traffic light AND the lights indicate red, THEN stop.

Examples: Production System

45

- Let's consider a simple example of a production system in the context of a thermostat control system for a heating, ventilation, and air conditioning (HVAC) unit.
- The goal is to maintain a comfortable temperature in a room.
- **Global Database:**
 - ▣ Current room temperature (e.g., 72°F)
 - ▣ Desired temperature setpoint (e.g., 68°F)
 - ▣ Time of day (morning, afternoon, evening)

Examples: Production System

46

□ Set of Rules:

- Rule 1: IF (Current Temperature $<$ Desired Setpoint) THEN Turn On Heater
- Rule 2: IF (Current Temperature $>$ Desired Setpoint) THEN Turn On Air Conditioner
- Rule 3: IF (Time of Day = Evening) THEN Adjust Setpoint Up by 2°F
- Rule 4: IF (Time of Day = Morning) THEN Adjust Setpoint Down by 1°F

Examples: Production System

47

□ **Control System:**

- The control system monitors the global database and activates rules based on the conditions.
- During the morning, Rule 4 may be triggered, adjusting the setpoint down by 1 °F.
- If the current temperature is below the new setpoint, Rule 1 may be triggered to turn on the heater.
- As the day progresses, Rule 3 may adjust the setpoint up in the evening.
- If the room becomes too warm, Rule 2 may be triggered to turn on the air conditioner.

Examples: Production System

48

- **Morning:** Current Temperature = 70°F, Desired Setpoint = 68°F
 - ▣ Rule 4 triggered, setpoint adjusted down to 67°F
 - ▣ Rule 1 triggered, heater turned on to warm the room.
- **Afternoon:** Current Temperature = 67.5°F, Desired Setpoint = 67.5°F
 - ▣ No rule triggered, system maintains the current state.
- **Evening:** Current Temperature = 68.5°F, Desired Setpoint = 67°F
 - ▣ Rule 3 triggered, setpoint adjusted up to 69°F.
- **Night:** Current Temperature = 69.5°F, Desired Setpoint = 69°F
 - ▣ Rule 2 triggered, air conditioner turned on to cool the room.

Constraint Satisfaction Problem: Rules Crypto-arithmetic Problem

49

- Each Letter, Symbol represents only one digit throughout the problem.
- Numbers must not begin with zero i.e. 0567 (wrong) , 567 (correct).
- Aim is to find the value of each letter in the Cryptarithmic problems
- There must be only one solution to the Cryptarithmic problems
- The numerical base, unless specifically stated , is 10.
- After replacing letters by their digits, the resulting arithmetic operations must be correct.
- Carry over can only be 1 in Cryptarithmic problems

Constraint Satisfaction Problem: Crypto-arithmetic Problem

50

- SEND+MORE=MONEY
- TO + GO = OUT
- DONALD+HERALD=ROBERT
- CROSS+ROADS=DANGER

$$\begin{array}{r} 96233 \\ +62513 \\ \hline 158746 \end{array}$$

- WRONG+WRONG=RIGHT

SEND+MORE=MONEY : Solution

51

$$D+E=Y+10.x1$$

$$x1+N+R=E+10.x2$$

$$x2+E+O=N+10.x3$$

$$x3+S+M=O+10.x4$$

$$x5=M$$

1. $M=1$, since carry from two digits and another carry can't total more than 19

2. S can be 8 or 9 since $S+M+x3>9$ to generate carry
 $M=1 : S+1+x3>9 \rightarrow S+x3>8$
3. $O=0$, since $S+M+x3$ must be at least 10 and at most 11
But $M=1$ so, $O=0$
4. $N=E$ or $N=E+1$
Since, $N \neq E$, $N=E+1$

SEND+MORE=MONEY : Solution

52

$$N=E \text{ or } N=E+1$$

$$\text{as } N \neq E, N=E+1$$

$$\text{Let } S=9 \rightarrow x_3=0$$

Choose E as first choice for variable selection since it appears most.

$$x_1+N+R=E+10 \quad x_2$$

$$\text{Let } E=6 \text{ and } x_1=1, N=7$$

$$1+7+R=16$$

$$\text{Hence, } R=8$$

$$\text{Also, } D+E=Y+10$$

$$D+6=Y+10$$

Hence,

$$S=9, E=5, N=6, D=7, \\ M=1, O=0, R=8, Y=2$$

Game Playing

53

- ❑ Game Playing is an important domain of artificial intelligence.
- ❑ Games don't require much knowledge; ***the only knowledge we need to provide is the rules, legal moves and the conditions of winning or losing the game.***
- ❑ Both players try to win the game. So, both of them try to make the best move possible at each turn.
- ❑ Searching techniques like BFS(Breadth First Search) are not accurate for this as the branching factor is very high, so searching will take a lot of time.
- ❑ So, we need another search procedures that improve :
 - ▣ Generate procedure so that only good moves are generated.
 - ▣ Test procedure so that the best move can be explored first.

Game Playing

54

- The most common search technique in game playing is Minimax search procedure.
- It is depth-first depth-limited search procedure. It is used for games like chess and tic-tac-toe.
- **Minimax algorithm uses two functions :**
 - ▣ **MOVEGEN:** It generates all the possible moves that can be generated from the current position.
 - ▣ **STATIC EVALUATION:** It returns a value depending upon the goodness from the viewpoint of two-player
- This algorithm is a two player game, so we call the first player as PLAYER1 and second player as PLAYER2.

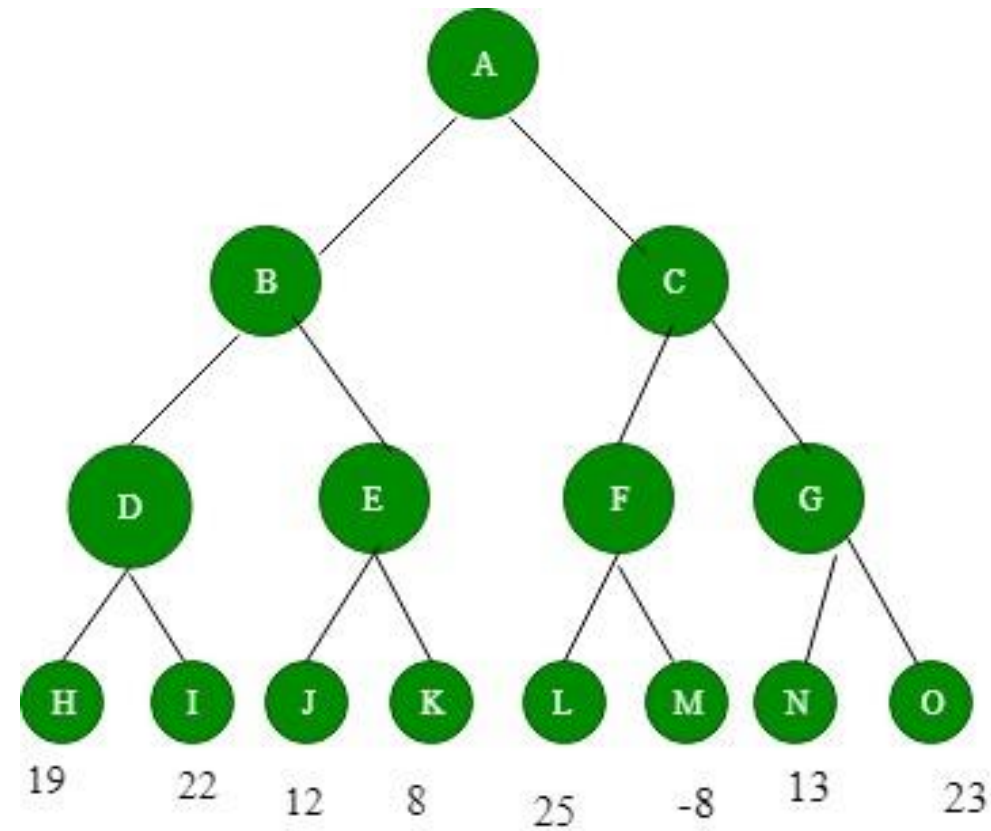
Game Playing

55

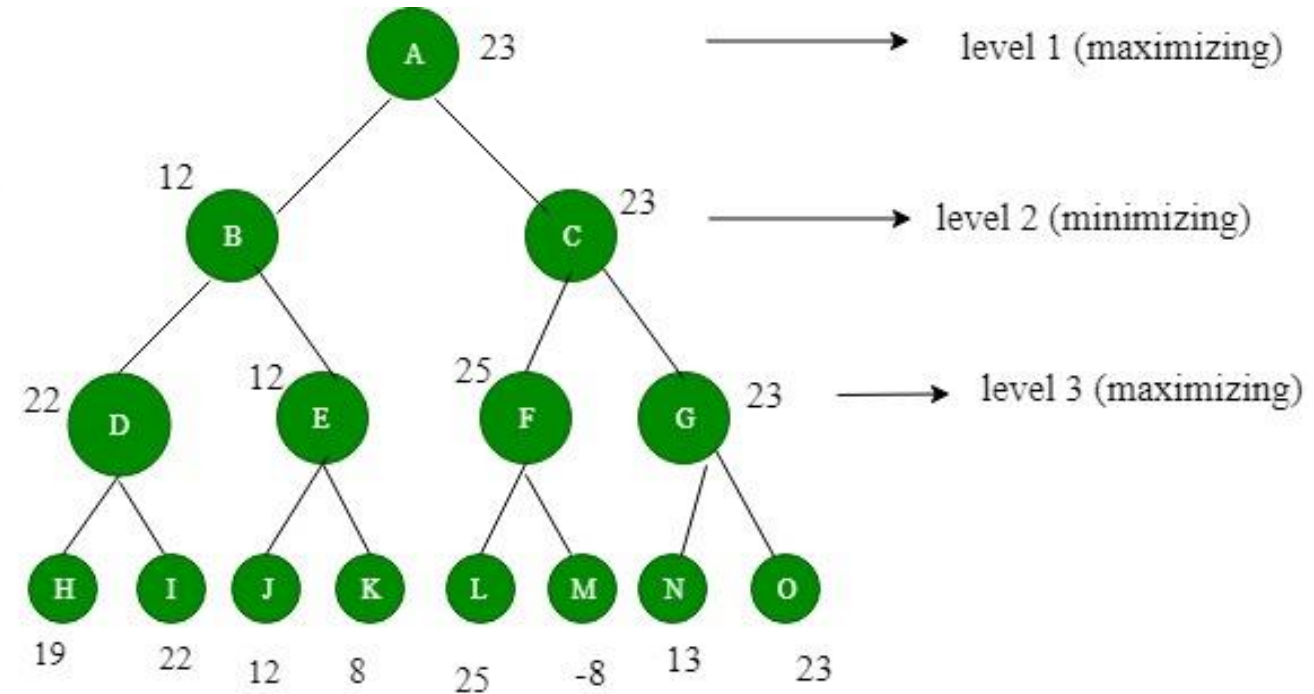
- The value of each node is backed-up from its children. For PLAYER1 the backed-up value is the maximum value of its children and for PLAYER2 the backed-up value is the minimum value of its children.
- It provides most promising move to PLAYER1, assuming that the PLAYER2 has make the best move.
- It is a recursive algorithm, as same procedure occurs at each level.
- We assume that PLAYER1 will start the game. 4 levels are generated.
- The value to nodes H, I, J, K, L, M, N, O is provided by STATICEVALUATION function.

Game Playing

56



Backing-up
of values



Game Playing

57

- Level 3 is maximizing level, so all nodes of level 3 will take maximum values of their children.
- Level 2 is minimizing level, so all its nodes will take minimum values of their children.
- This process continues. The value of A is 23. That means A should choose C move to win.

Game Playing

58

- Multi agent environments
- Games and adversarial search [Unit3 → 3.1]
- Game theory: a branch of economics that views any multi agent environment as a game provided that the impact of each agent on the others is significant, regardless of whether the agents are cooperative and competitive

References

59

- Russell, S. and Norvig, P., 2011, Artificial Intelligence: A Modern Approach, Pearson, India.
- Rich, E. and Knight, K., 2004, Artificial Intelligence, Tata McGraw hill, India.

60

Thank You

Any Queries?

Now Try to Solve your Problems.